

Performance improvement

■ Fixed cycle time

- Ⓜ Obviously inefficient

■ Variable cycle time

- Ⓜ Faster (in the example only 2/3 of the time)
- Ⓜ Rather complex timing

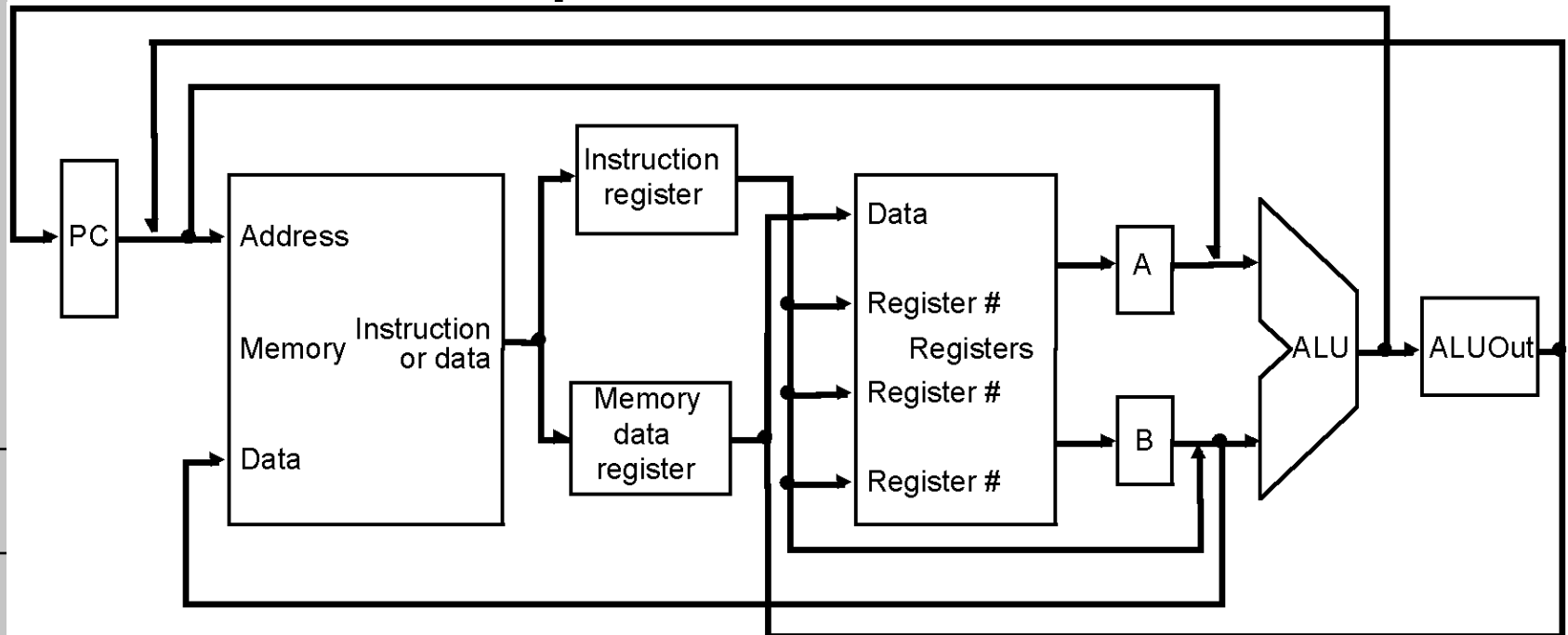
■ Multiple clock cycles

- Ⓜ Break up the instructions into one-cycle steps
- Ⓜ Balance the amount of work to be done
- Ⓜ Restrict each cycle to use only one major functional unit
- Ⓜ Store values at the end of each cycle for use in later cycles (easiest thing to do)

Multiple clock cycle implementation

■ Buffer registers updated each cycle

- Ⓡ Data
- Ⓡ Register file outputs
- Ⓡ ALU output
- Ⓡ Instruction, updated at end of instruction



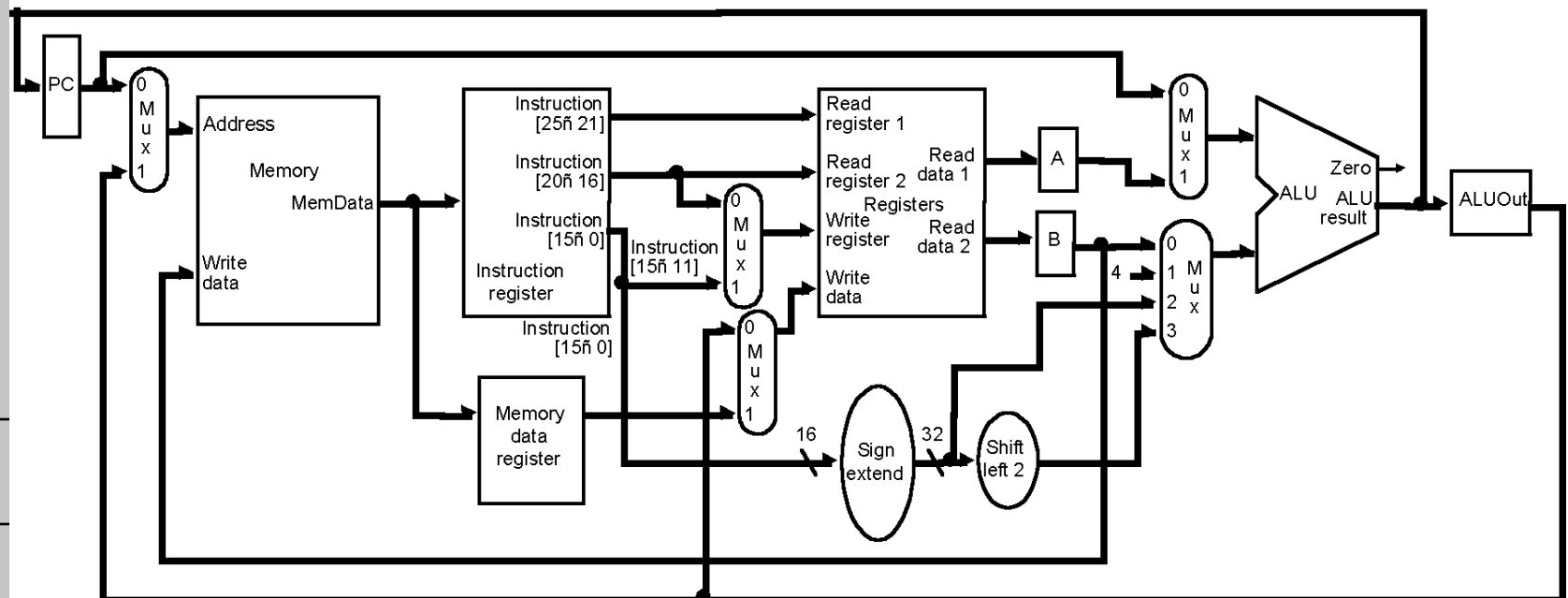
Data path

■ Use of the ALU for

- Ⓡ PC increment
- Ⓡ Branch
- Ⓡ Memory address
- Ⓡ Arithmetic operations

■ Additional data paths

■ Interpretation of IR depends on instruction format



Data path with control signals

① Memory address

① Memory R/W

① IR write

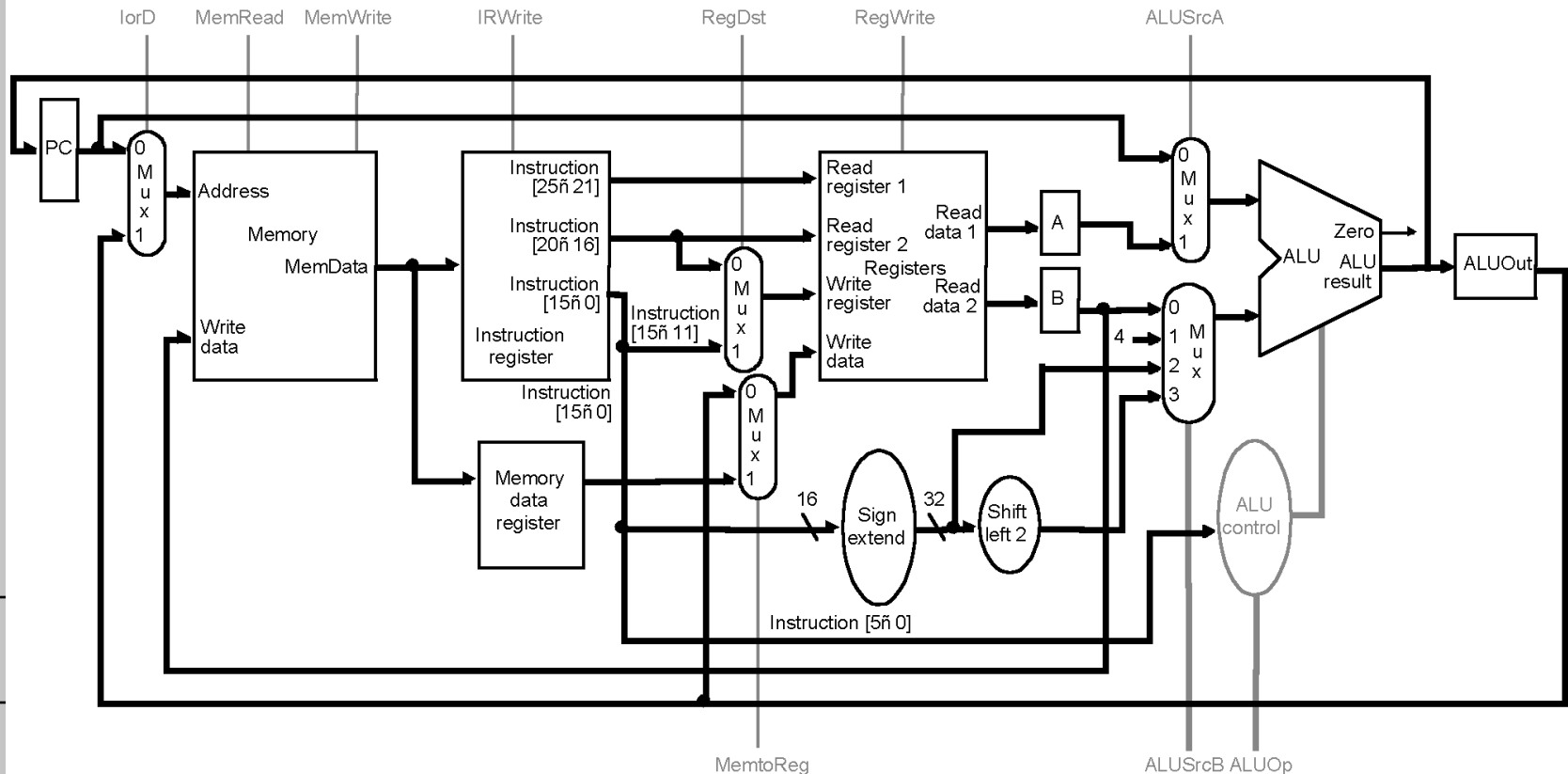
① Register write

① Write register destination

① Write register data source

① ALU source multiplexors

① ALU control



PC handling

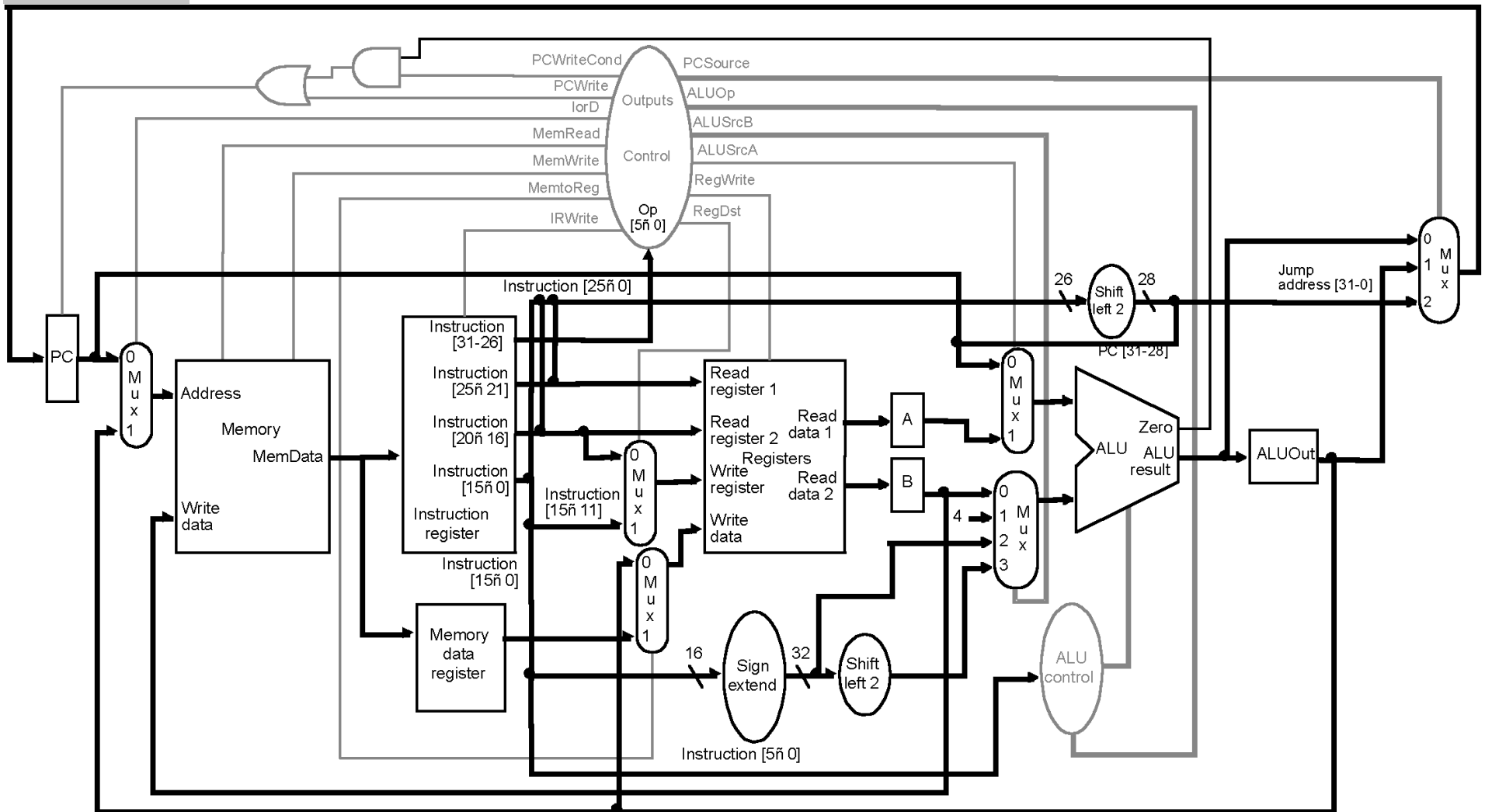
■ Multiplexor selecting the source for the PC

- ① ALU output after $PC + 4$ (normal instruction)
- ① ALUOut after branch target address calculation
- ① New address with a jump instruction

■ Writing the PC

- ① Unconditionally after a normal instruction and jump
- ① Conditionally overwritten if a branch is taken
 - | PCWriteCond allows conditional loading
 - | Zero bit from ALU decides if the PC is reloaded with the branch target address

Complete data path & control



Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register file destination number for the Write register comes from the rt field.	The register file destination number for the Write register comes from the rd field.
RegWrite	None	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None	Memory contents at the location specified by the Address input is replaced by value on Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
lorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None	The output of the memory is written into the IR.
PCWrite	None	The PC is written; the source is controlled by PCSource.
PCWriteCond	None	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the sign-extended, lower 16 bits of the IR.
	11	The second input to the ALU is the sign-extended, lower 16 bits of the IR shifted left 2 bits.
PCSource	00	Output of the ALU ($PC + 4$) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address ($IR[25-0]$ shifted left 2 bits and concatenated with $PC + 4[31-28]$) is sent to the PC for writing.

Breaking the instructions

■ Move from one-cycle to multi-cycle

- Ⓜ Identifying steps that take one cycle
- Ⓜ Equal distribution of execution time
- Ⓜ At most one operation for each of the modules
 - | ALU
 - | Register file
 - | Memory

■ New registers if

- Ⓜ The signal is computed in one cycle and used in another cycle
- Ⓜ The inputs of the block generating the signal may change in the second cycle

The five execution steps

■ Instruction fetch

- Ⓜ Move the instruction from the instruction memory to the instruction register IR

■ Instruction decode and register fetch

- Ⓜ Provide the register contents for the ALU

■ Execution, memory address computation or branch completion

■ Memory access or R-type instruction completion

■ Write back step

Step 2: Instruction decode & register fetch

- Switch registers to the output of the register block

A = register [IR [25-21]] rs

B = register [IR [20-16]] rt

Ⓡ No signal setting required

- Calculate the branch target address
target = PC + (sign-ext. (IR [15-0]) << 2)

Ⓡ Stored in the ALUOut register

Ⓡ Set ALUSrcB = 11

Ⓡ Set ALUOp = 00 add

Step 3: Execution, memory address computation or branch completion

- Step depends on the instruction
- Selection performed by interpretation of the op + function field of the instruction
- Calculate memory reference address

$$\text{ALUOut} = A + \text{sign-ext. (IR[15-0])}$$
 - Ⓡ Set ALUSrcA = 1 get operand from A
 - Ⓡ Set ALUSrcB = 10 get operand from sign extension unit
 - Ⓡ Set ALUOp = 00 add

Step 3: Execution, memory address computation or branch completion

■ Arithmetic-logical instruction (R-type)

$ALUOut = A \text{ op } B$

- Ⓡ Set $ALUSrcA = 1$ get operand from A
- Ⓡ Set $ALUSrcB = 00$ get operand from B
- Ⓡ Set $ALUOp = 10$ code from IR

■ Branch: if (A == B) $PC = ALUOut$

- Ⓡ Set $ALUSrcA = 1$ get operand from A
- Ⓡ Set $ALUSrcB = 00$ get operand from B
- Ⓡ Set $ALUOp = 01$ subtraction
- Ⓡ Write $ALUOut$ to PC register

■ Jump: $PC = PC[31-28] \parallel (IR[25-0] \ll 2)$

4: Memory access or R-type instruction completion

■ Memory access

Ⓡ ALU controls must remain stable

Ⓡ Set I or D = 1 address from ALU

memory-data = memory [ALUOut]
load from memory

Ⓡ Set MemRead = 1

memory [ALUOut] = B
store to memory

Ⓡ Set MemWrite = 1

4: Memory access or R-type instruction completion

- Arithmetic-logical instruction completion
Register [IR [15-11]] = ALUOut

- ④ Set RegDst = 1 Select write register
- ④ Set RegWrite = 1 Allow write operation
- ④ Set MemToReg = 0 Select ALU data
- ④ ALUOp, ALUSrcA, ALUSrcB = constant

5: Write back

- Write data from memory to the register

Reg [IR[20-16]] = memory-data

- ① Set RegDst = 0 Select write rt
as target register
- ① Set RegWrite = 1 Allow write operation
- ① Set MemToReg = 1 Select Memory data
- ① ALUOp, ALUSrcA, ALUSrcB = constant

Execution step summary

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	$IR = \text{Memory}[PC]$ $PC = PC + 4$			
Instruction decode/register fetch	$A = \text{Reg}[IR[25-21]]$ $B = \text{Reg}[IR[20-16]]$ $ALUOut = PC + (\text{sign-extend}(IR[15-0]) \ll 2)$			
Execution, address computation, branch/jump completion	$ALUOut = A \text{ op } B$	$ALUOut = A + \text{sign-extend}(IR[15-0])$	if $(A == B)$ then $PC = ALUOut$	$PC = PC[31-28] \parallel (IR[25-0] \ll 2)$
Memory access or R-type completion	$\text{Reg}[IR[15-11]] = ALUOut$	Load: $MDR = \text{Memory}[ALUOut]$ or Store: $\text{Memory}[ALUOut] = B$		
Memory read completion		Load: $\text{Reg}[IR[20-16]] = MDR$		

■ Five steps:

① Common steps

② Instruction specific steps

Control Signals

Signal Name	Value	Effect
MemRead	0	None
	1	Contents of memory at the read address are output on read data out
MemWrite	0	None
	1	Memory contents at write address is replaced by value on data output
ALUSrcA	0	The first ALU operand is the PC
	1	The first ALU operand comes from the register given by the rs field
ALUSrcB	00	The second ALU operand comes from the register given by the rt field
	01	The second ALU operand is the constant 4
	10	The second ALU operand is the sign-extended bits [15-0] of IR
	11	The second ALU operand is the sign-extended & shifted bits [15-0] of IR
ALUOp	00	Addition operation
	01	Subtraction operation
	10	Function depends on the contents of the code field in the IR
	11	Undefined
RegDst	0	The destination register comes from the rt field of the IR
	1	The destination register comes from the rd field of the IR
RegWrite	0	None
	1	Enables writing the register given by the write register number
MemtoReg	0	The value for the written register comes from the ALU
	1	The value for the written register comes from data memory
lorD	0	The memory address comes from the PC
	1	The memory address comes from the ALU
IRWrite	0	None
	1	The value from memory is written into IR
PCWrite	0	
	1	PC write, source is defined by PCSource
PCWriteCond	0	
	1	PC write IF ALU indicates a ZERO condition
PCSource	00	PC = ALU output
	01	PC = target register
	10	PC = jump target address (PC + 4 [29-26] & IR [25-0] <<2
	11	undefined

Controlling the processor

- Time dependent sequence of steps
-> Finite State Machine
- Description methods
 - Ⓜ State diagram
 - Ⓜ Microprogramming
 - Ⓜ VHDL
 - Ⓜ Program
 - Ⓜ ...
- In this example: State diagram & Microprogram

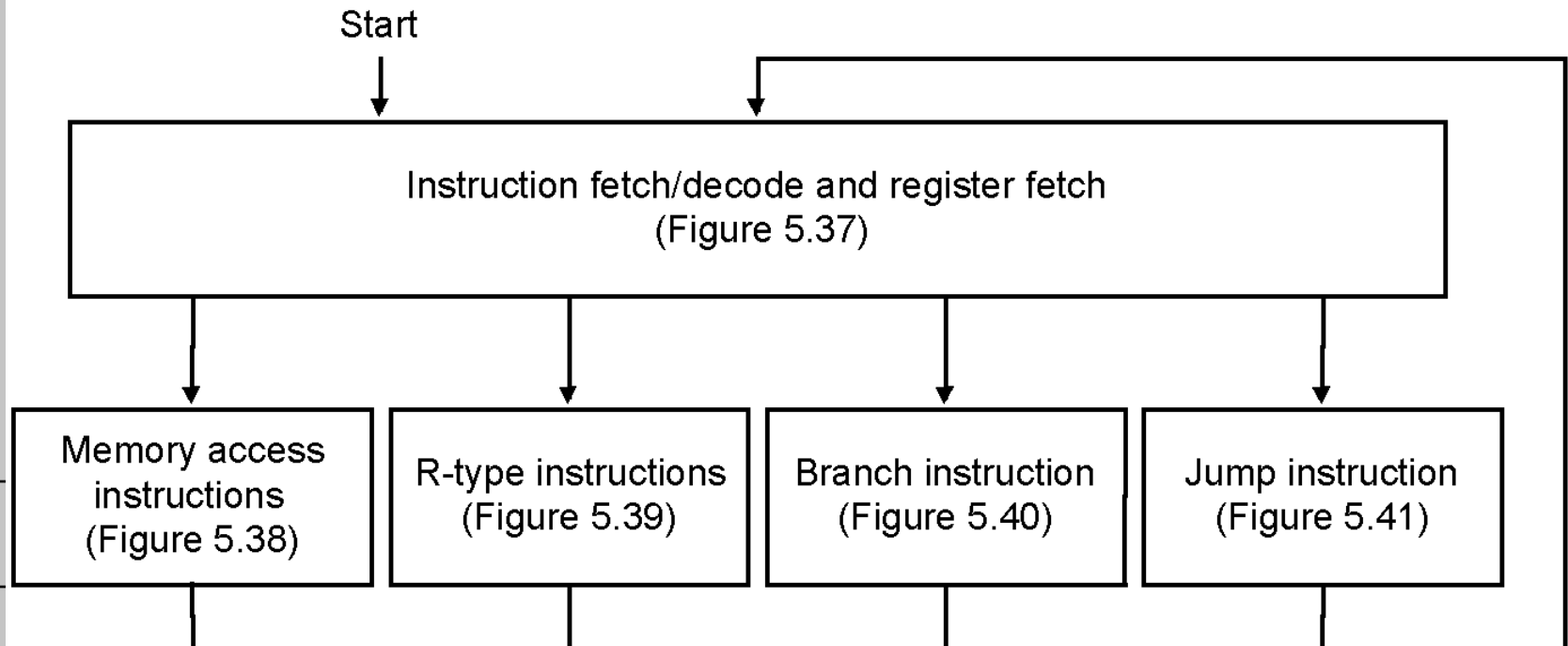
General decomposition

■ Common part

® Instruction fetch

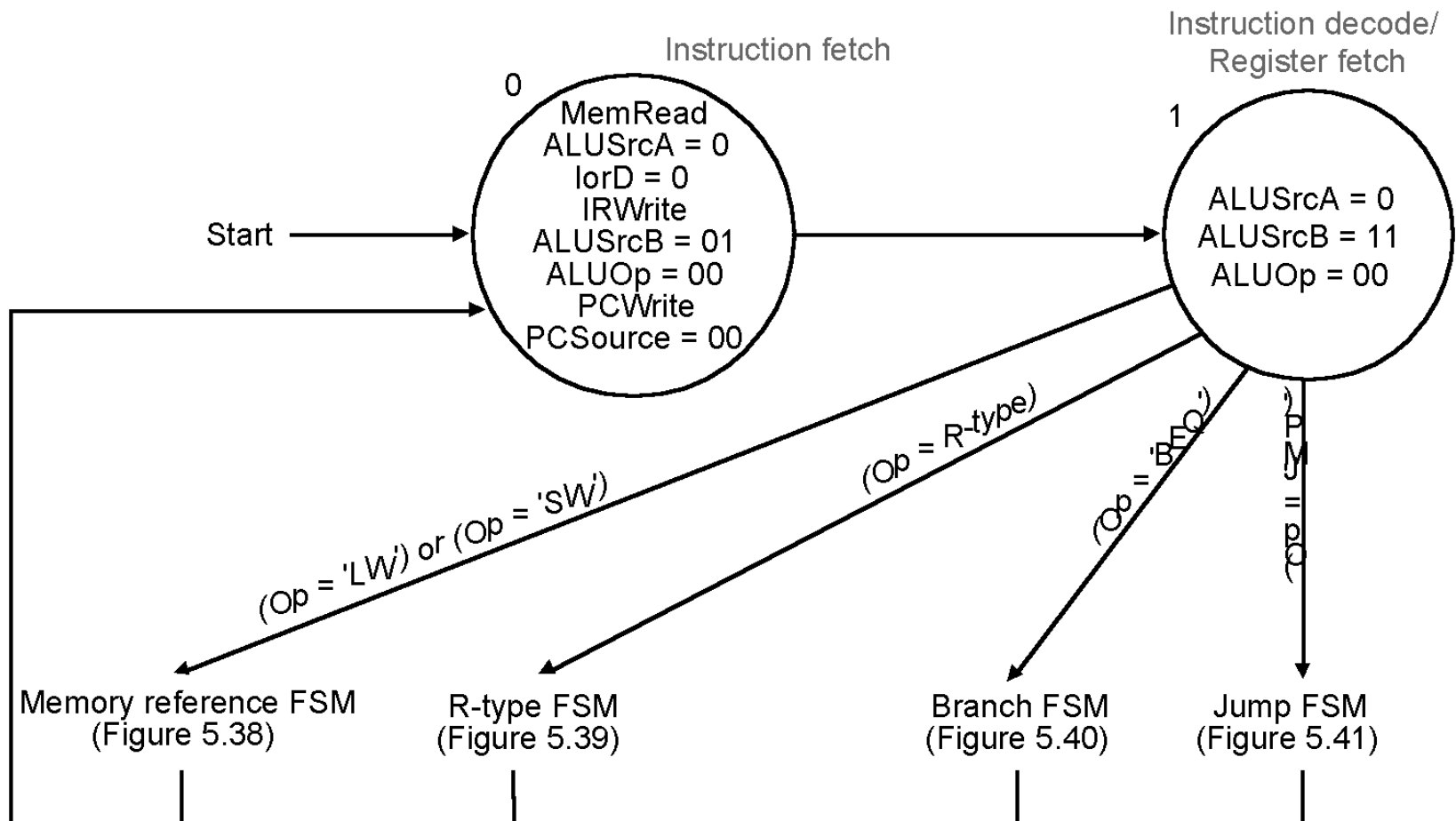
® Instruction decode/register fetch

■ Instruction specific part



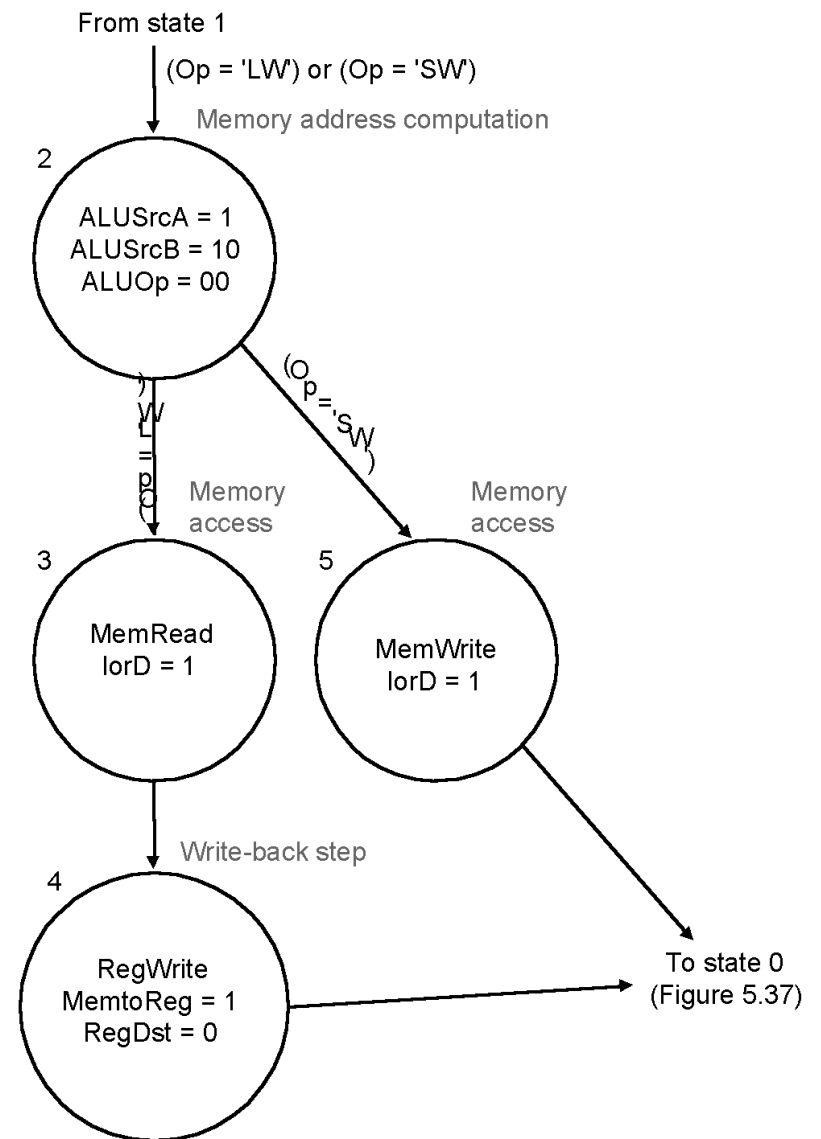
Common part

- Instruction fetch
- Instruction decode/register fetch



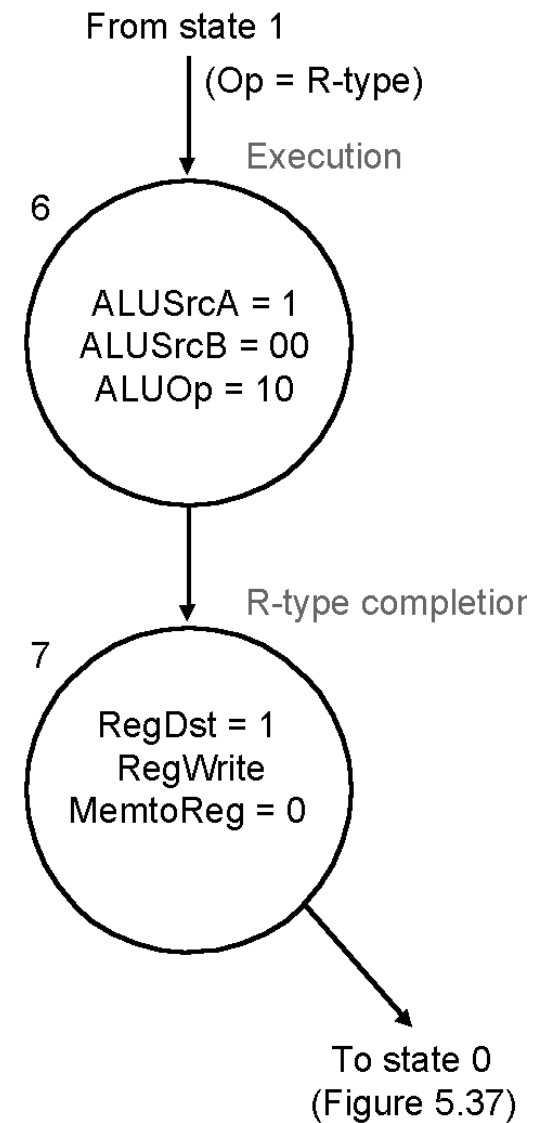
Memory access

- Address calculation
- Load sequence
 - Ⓜ Read from memory
 - Ⓜ Store to register
- Access memory
- Store sequence
Write



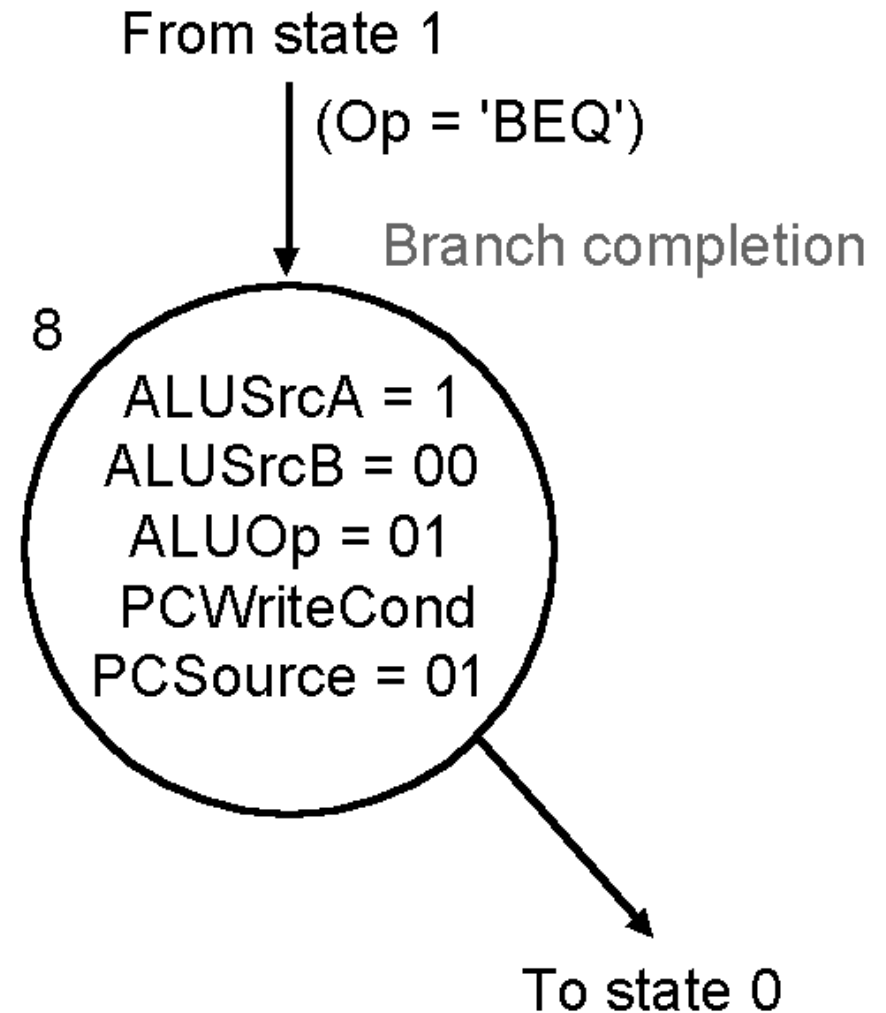
R-type instruction

- Execution of instruction
- Completion
Write result to register



Branch instruction

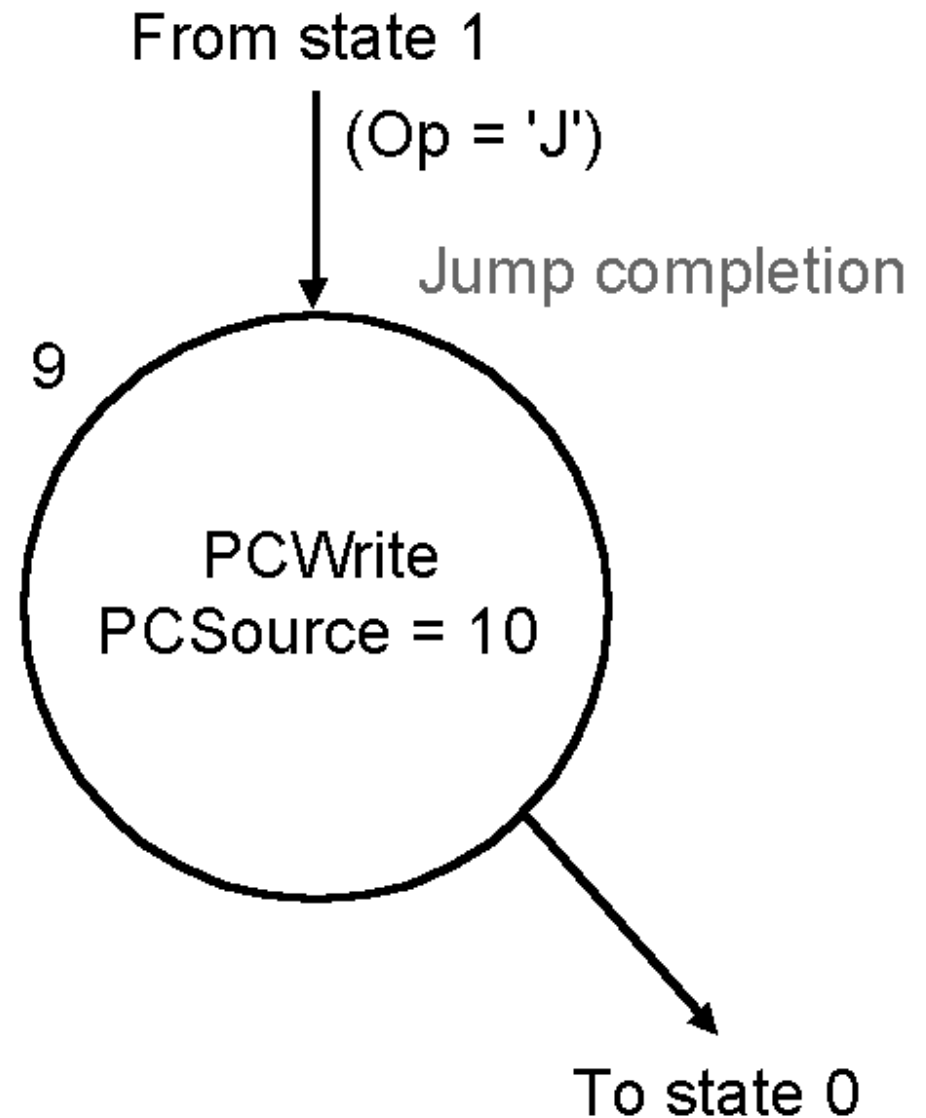
- Single state
- PC is loaded with branch address
- Get next instruction



Jump instruction

- Load PC with new value
- Get next instruction

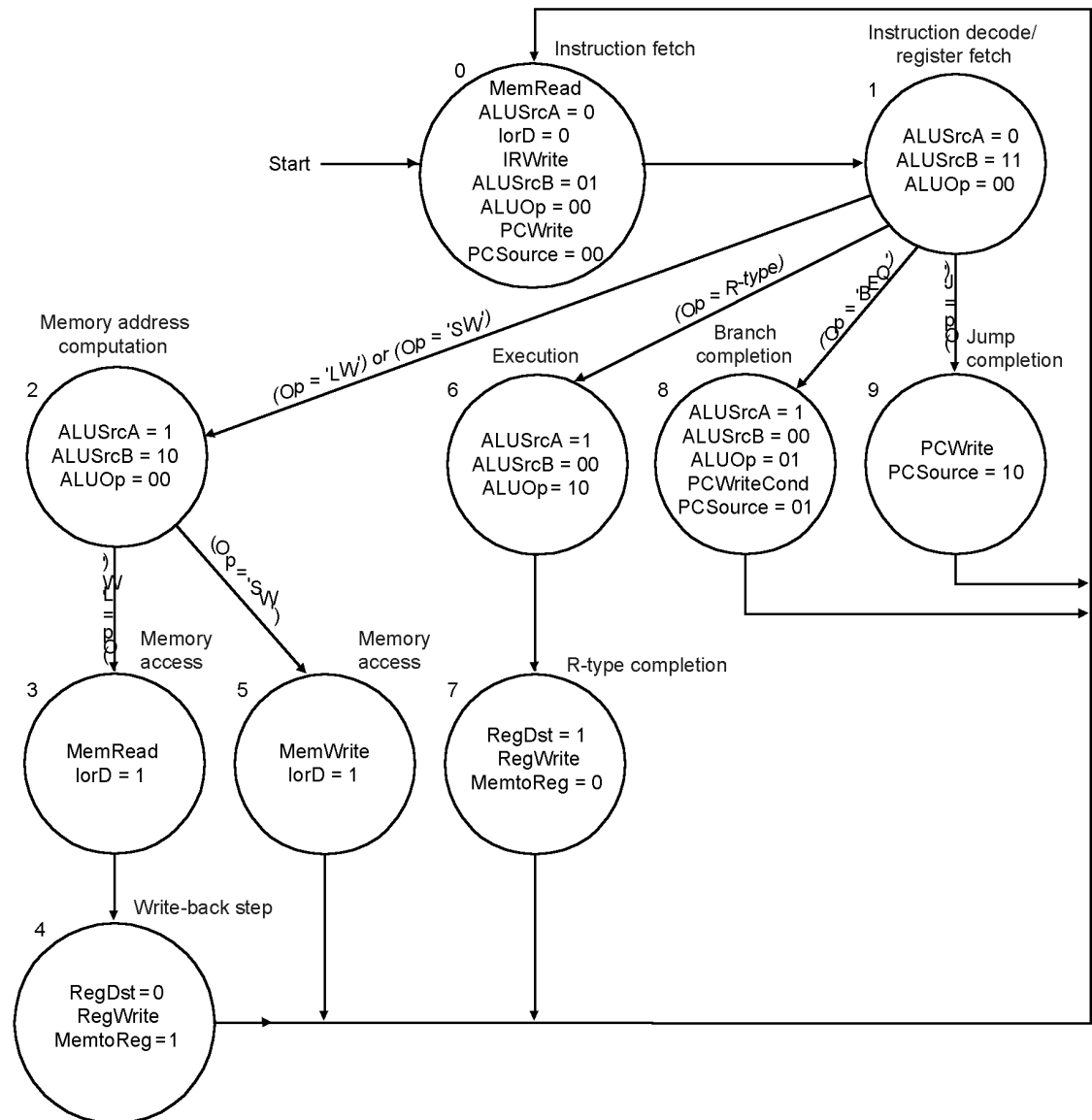
The Processor



Complete state machine

Common part

Instruction specific part



Control signal summary

The Processor

					States					
Signal Name	0	1	2	3	4	5	6	7	8	9
MemRead	1	0	0	1	1	0	0	0	0	0
MemWrite	0	0	0	0	0	1	0	0	0	0
ALUSrcA	0	0	1	1	1	1	1	1	1	-
ALUSrcB	01	11	10	10	10	10	00	00	00	-
ALUOp	00	00	00	00	00	00	10	10	01	-
RegDst	-	-	-	-	0	-	-	1	-	-
RegWrite	0	0	0	0	1	0	0	1	0	0
MemtoReg	-	-	-	-	1	-	-	0	-	-
lrd	0	-	-	1	1	1	-	-	-	-
IRWrite	1	0	0	0	0	0	0	0	0	0
PCWrite	1	0	0	0	0	0	0	0	0	1
PCWriteCond	0	0	0	0	0	0	0	0	1	0
TargetWrite	0	1	0	0	0	0	0	0	0	0

! Black: active signals

! Red: inactive signals that must be set

! dash: don't care

Performance

■ Instruction mix & states

Instruction	Length	Benchmark
Loads	5	22%
Stores	4	11%
R-format	4	49%
Branch	3	16%
Jump	3	2%

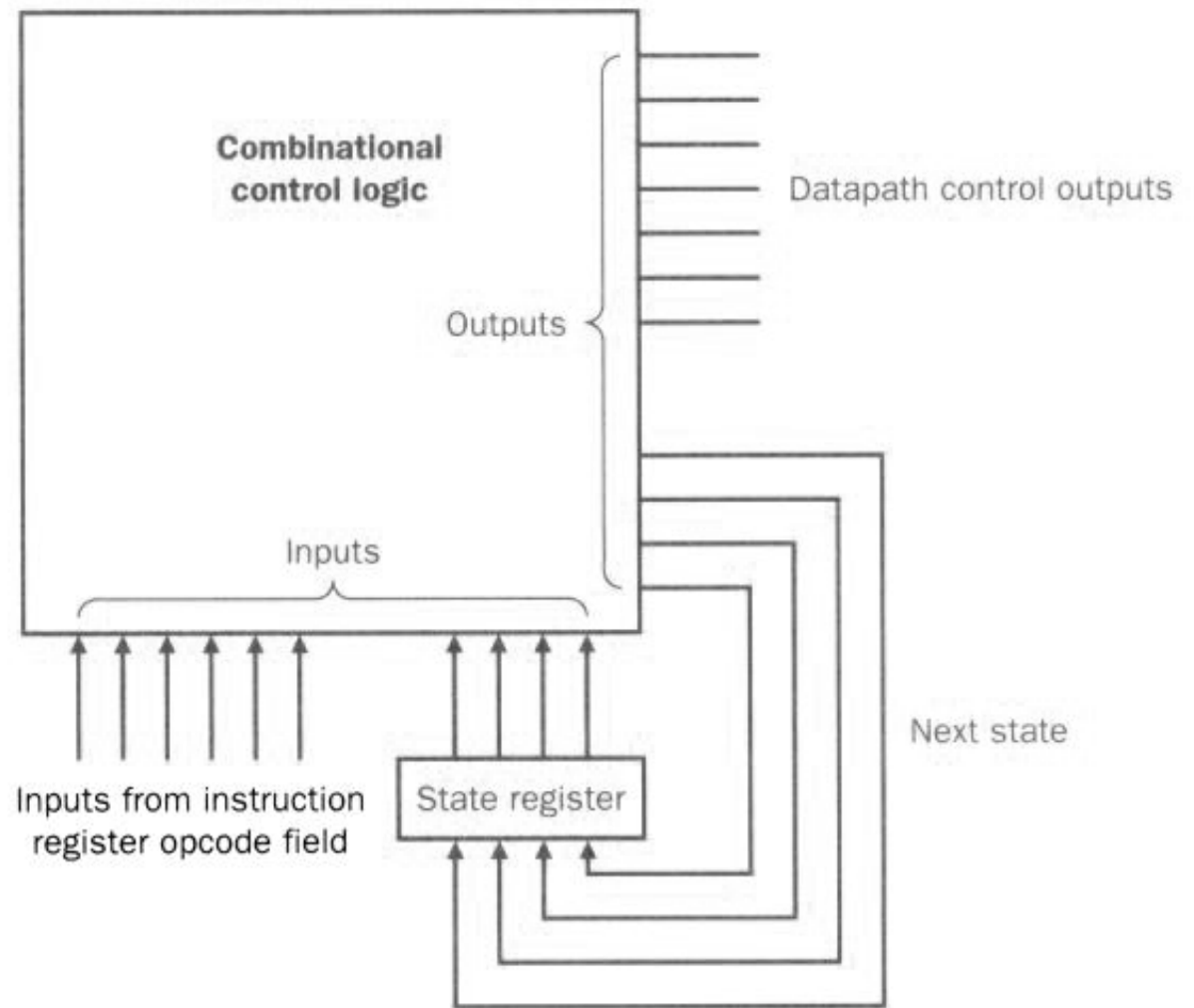
■ CPI

$$0.22 \cdot 5 + 0.11 \cdot 4 + 0.49 \cdot 4 + 0.16 \cdot 3 + 0.02 \cdot 3$$

$$= 4.033$$

Implementation possibilities

- **Moore machine:**
The outputs depend on inputs and state



Implementation options

■ ROM

④ Address bits: 10

┆ State register: 4 bits

┆ Opcode field: 6 bits

④ Output bits: 17

■ PLA

④ Calculation of the output functions

④ Optimisation

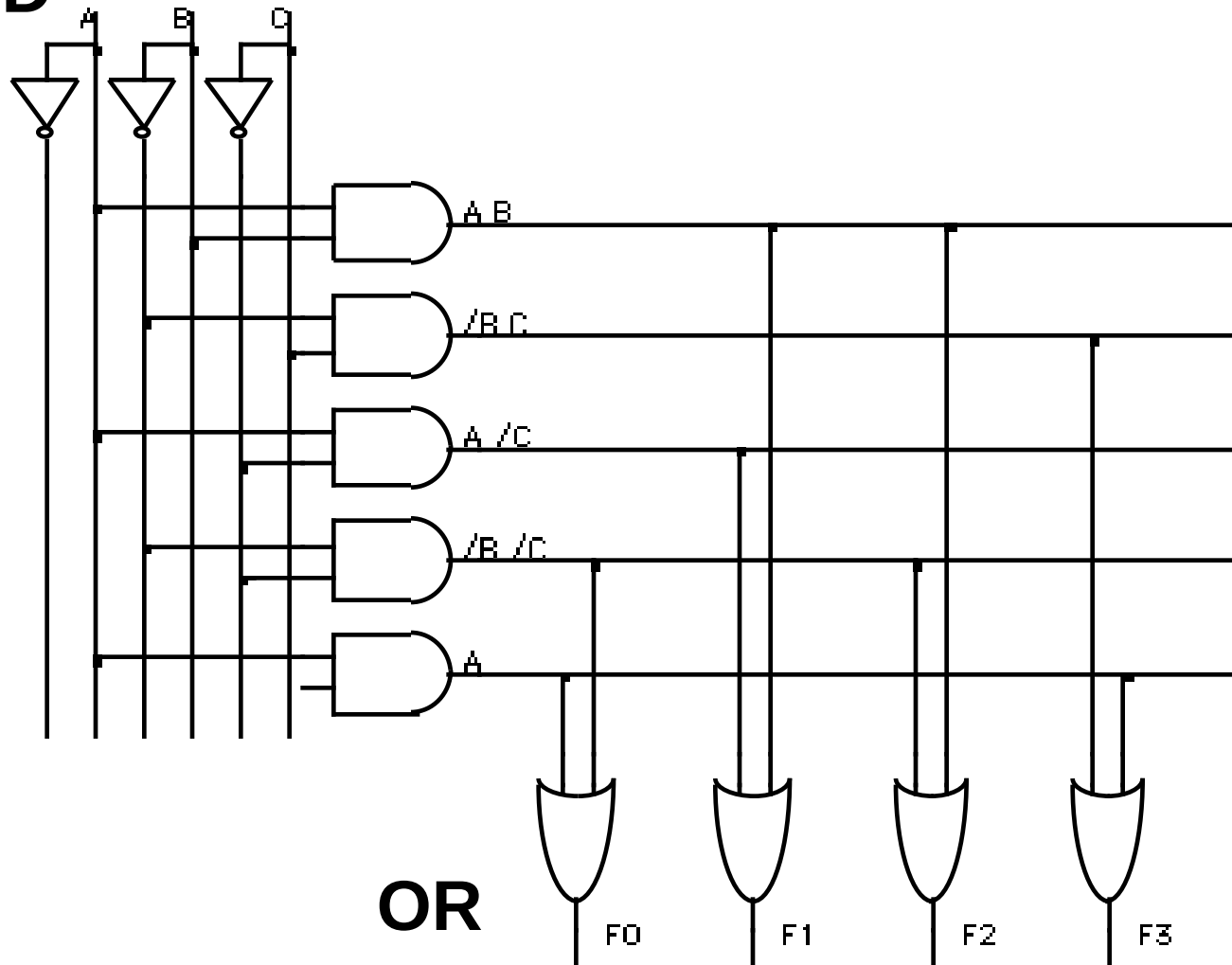
④ Mapping to PLA architecture

④ Smaller area

④ Faster implementation

PLA

■ AND



OR

Microprogramming

- **Limitation of FSM's based on state diagrams is the complexity**
 - Ⓡ **MIPS: more than 100 instructions**
 - Ⓡ **Instruction length: 1 - 20 cycles**
 - Ⓡ **Number of states: enormously growing**
- **One alternative: Microprogramming**
 - Ⓡ **Implementation of the MIPS instructions as a sequence of simpler instructions**
 - Ⓡ **Design of the micro instruction format**
 - Ⓡ **Writing the program**

Micro instructions

■ What do we need?

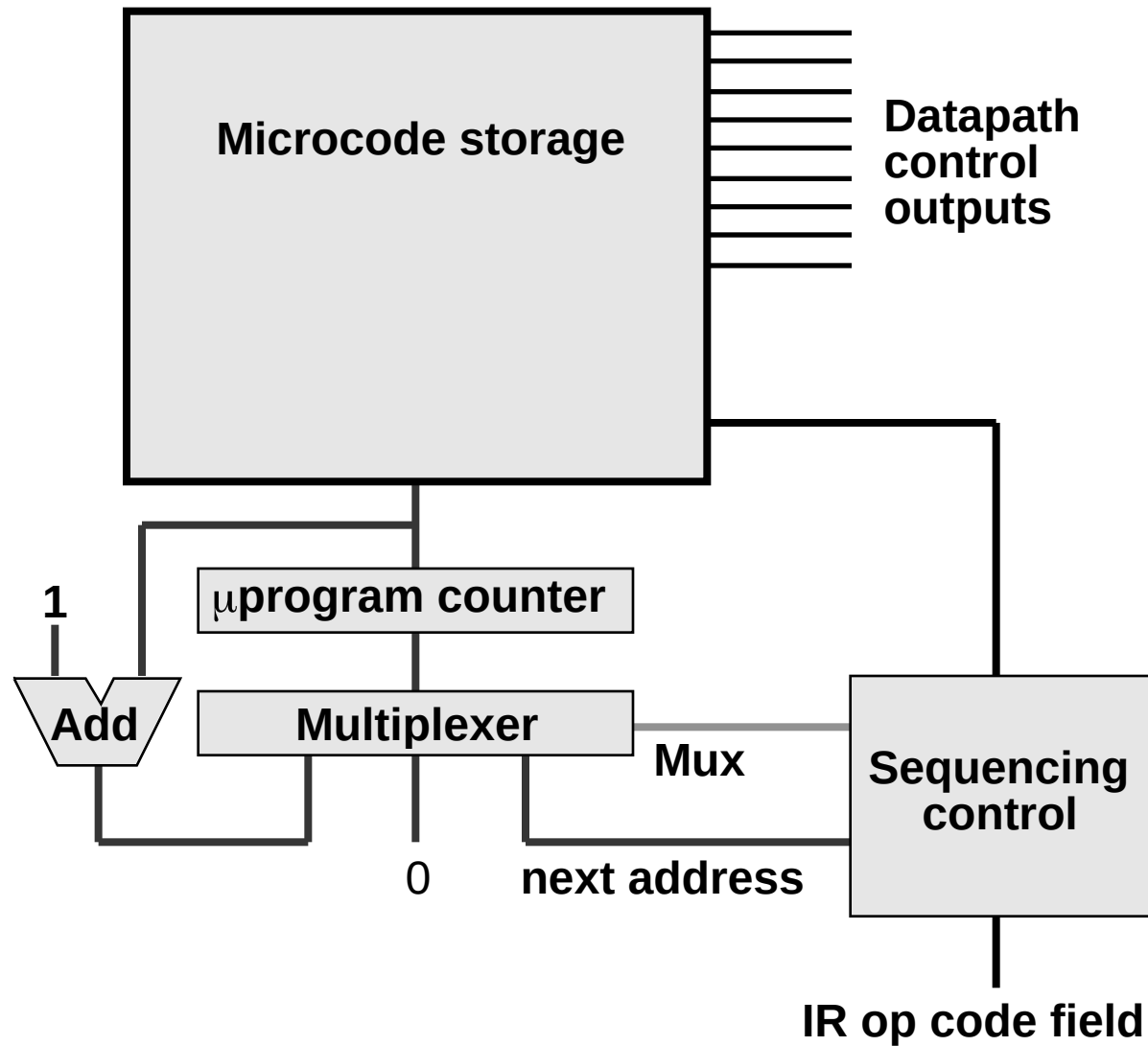
- ① ALU control signals
- ① Program counter control signals
- ① Complete data path

■ What do we do?

- ① Analyse what is happening at each step
- ① Write it down in a column oriented way
- ① Translate it to binary
- ① Store it in an appropriate form

A simple implementation

The Processor



Elements

■ Microcode storage (ROM or PLA)

- Ⓡ Datapath control outputs

- Ⓡ Sequencing control out

■ Output decoder (optional)

■ Microprogram counter / Mux

- Ⓡ Reset

- Ⓡ Increment

- Ⓡ New address

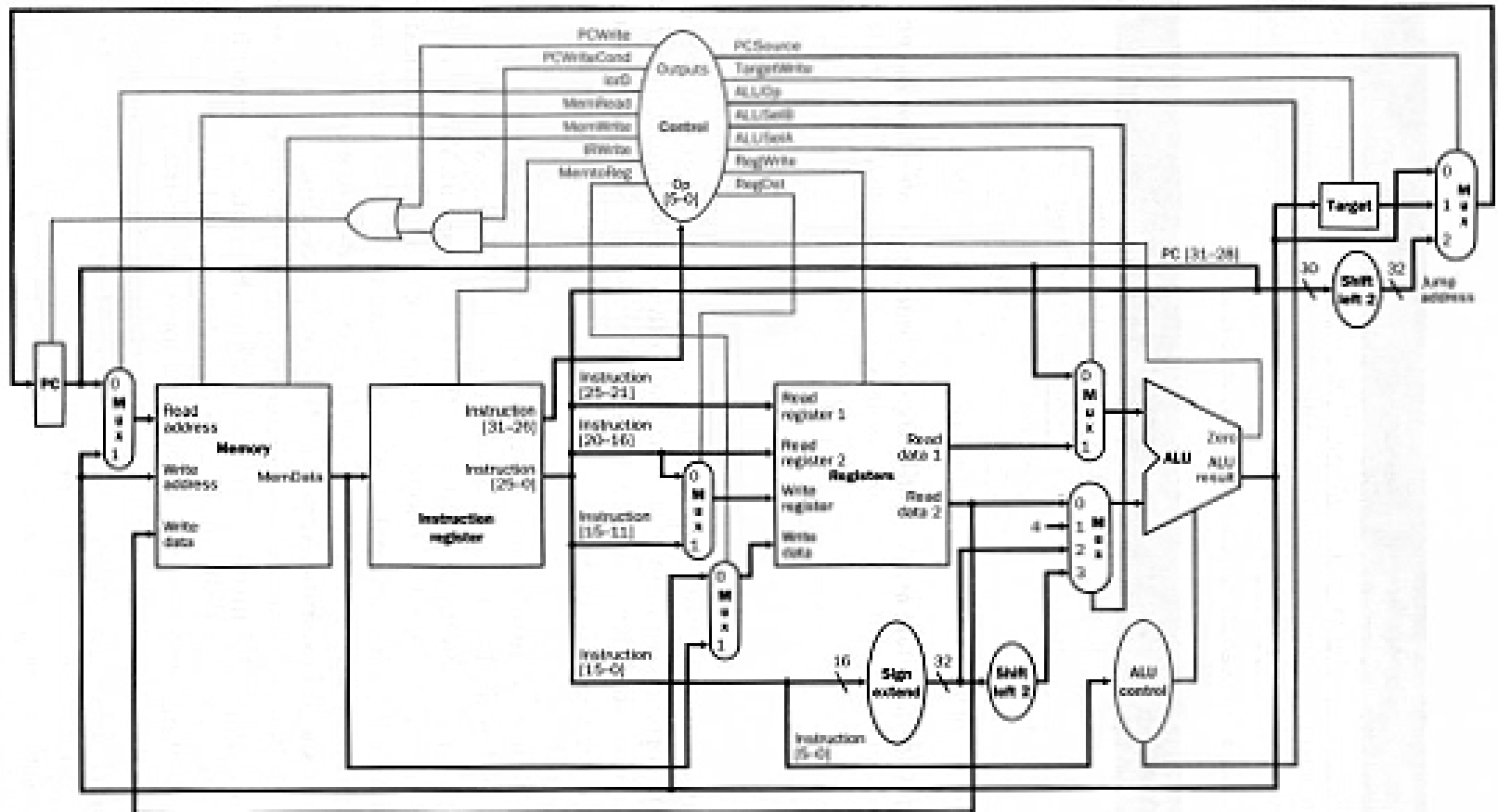
■ Sequencing control

- Ⓡ Mux control

- Ⓡ Next address calculation

Data path

■ Data path and control



Micro instruction format

- Sort the signals according to their function
- Signals that are never used at the same time can be shared
- These signals have to be set !

Field name	Function of field
ALU control	Specify the operation being done by the ALU during this clock; the result is always written in ALUOut.
SRC1	Specify the source for the first ALU operand.
SRC2	Specify the source for the second ALU operand.
Register control	Specify read or write for the register file, and the source of the value for a write.
Memory	Specify read or write, and the source for the memory. For a read, specify the destination register.
PCWrite control	Specify the writing of the PC.
Sequencing	Specify how to choose the next microinstruction to be executed.

Microprogram

- Physical implementation: ROM or PLA
- Each micro instruction has an address
- Sequentially ordered
- Each sequence step is one cycle
- Selection of next instruction
 - Ⓡ Address increment (sequencing field = seq)
 - Ⓡ Fetch: begins the fetch of the next MIPS micro instruction (sequencing field = fetch)
 - Ⓡ Dispatch: jump to the next micro instruction the number i indicates the location in the dispatch table.

Control Signals

Field Name	Function	Effect
ALU Control	Add	Addition operation
	Subt	Subtraction operation
	Func code	Function depends on the contents of the code field in the IR
SRC 1	PC	Use the PC as the first ALU input.
	A	Register A is the first ALU input.
SRC 2	B	Register B is the second ALU input.
	4	The second ALU operand is the constant 4
	Extend	The second ALU operand is the sign-extended bits of IR
	Extshft	The second ALU operand is the sign-extended & shifted bits of IR
	Read	Read two registers using rd field of the IR and place the contents in A & B.
Register control	Write ALU	Write Register with data from ALU
	Write MDR	Write Register with data from the Memory Data Register
Memory	Read PC	Read memory using the PC address
	Read ALU	Read memory using the ALU output as address
	Write ALU	Write memory using the ALU output as address
PCWrite control	ALU	Write to the PC the ALU output
	Target cond	Write to the PC the contents of target IF Zero = active
	jump address	Write to the PC the jump address from instruction
Sequencing	Seq	Choose the next microinstruction
	Fetch	Go to the first microinstruction and begin a new instruction

Creating the microprogram

■ Start of each instruction

- ④ Fetch instruction
- ④ Calculate next PC
- ④ Decode instruction
- ④ Calculate branch PC

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Fetch	Add	PC	4		ReadPC	ALU	Seq
	Add	PC	Extshft	read			Dispatch 1

④ Blank field

- I Multiplexor: we don't care
- I Control signal for register access or function: it is set to 0

The start sequence

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Fetch	Add	PC	4		ReadPC	ALU	Seq
	Add	PC	Extshft	read			Dispatch 1

① First instruction

Fields	Effect
ALU control, SRC 1, SRC 2	Compute PC + 4
Memory	Fetch the instruction to IR
PCWrite control	Output of ALU is loaded into PC
Sequencing	Go to the next microinstruction

② Second instruction

Fields	Effect
ALU control, SRC 1, SRC 2	Store PC + sign extension (IR [15-0])
	<< 2 into ALUOut
Register control	Moves the data from the register file to A & B
Sequencing	Use dispatch table 1 for next address

Memory access

■ Microinstructions

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Mem 1	Add	A	Exend				Dispatch 2
LW2					Read ALU		seq
				Write MDR			Fetch
SW2					Write ALU		Fetch

■ Common part

Fields	Effect
ALU control, SRC 1, SRC 2	Memory address = rs + sign extend (IR[15-0])
	result available at the ALU output
Sequencing	Use 2nd dispatch tabel for continuation
	at SW 2 or LW 2

Memory access

■ Load word specific, 1

Fields	Effect
Memory	Read memory and write to MDR
Sequencing	Go to the next microinstruction

■ Load word specific, 2

Fields	Effect
Register control	Write the contents of the MDR to register rt
Sequencing	Go to the label Fetch

■ Store word specific

Fields	Effect
Memory	Write memory with ALUOut as address
Sequencing	Go to the label Fetch

R-format

■ Microinstruction

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Rfor1	FuncCode	A	B				seq
				Write ALU			Fetch

■ Operation

Fields	Effect
ALU control, SRC 1, SRC 2	ALU processes A & B according to function
Sequencing	Go to the next microinstruction

■ Finishing the operation

Fields	Effect
register control	Write the ALUOut int the register file
Sequencing	Go to the label Fetch

Branch & Jump

■ Branch

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
BEQ1	Subtr	A	B			AluOutC	Fetch

Fields	Effect
ALU control, SRC 1, SRC 2	ALU subtracts operands for Zero output
PC Write control	Load PC with ALUOut if Zero = 1
Sequencing	Go to the label Fetch

■ Jump

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Jump1						JpAdr	Fetch

Fields	Effect
PC Write control	PC loaded with jump target address

Complete microprogram

Label	ALU Con	SRC 1	SRC 2	RegCntl	Mem	PC Write	Sequence
Fetch	Add	PC	4		ReadPC	ALU	Seq
	Add	PC	Extshft	read			Dispatch 1
Mem 1	Add	A	Exend				Dispatch 2
LW2					Read ALU		seq
				Write Mem			Fetch
SW2					Write ALU		Fetch
Rfor1	FuncCode	A	B				seq
				Write ALU			Fetch
BEQ1	Subtr	A	B			AluOutC	Fetch
Jump1						JpAdr	Fetch

Untreated situations

■ Interrupts

- Ⓡ I/O device request

■ Exception handling

- Ⓡ Invocation of operating system from user
- Ⓡ Arithmetic overflow
- Ⓡ Undefined instruction
- Ⓡ Hardware malfunction

■ Error handling

- Ⓡ Recovery
- Ⓡ Abort operation
- Ⓡ Abort process
- Ⓡ reboot

Exception detection

■ Undefined instruction

- ④ Check the value of the op code field
- ④ On all values that do not belong to the valid set generate an interrupt

■ Arithmetic overflow

- ④ Is detected and generated by the ALU

■ Action

- ④ Save the PC of the current instruction (PC-4) in the exception program counter (EPC)
- ④ Continue with OS with error handling code
- ④ Return to the original code at the next instruction for restart

Exception handling

■ Version 1:

Use of a status register (cause register)

- ① Single entry point
- ① OS analyses the status register

■ Version 2:

Vectored interrupt

- ① For each interrupt cause a separate vector is stored
- ① Each interrupt vector targets at the appropriate piece of code

MIPS handling

■ Registers

Ⓡ Cause register, 32 bits

- | Individual bits are used for coding the cause, let's assume
- | Undefined instruction: bit 0 = 0
- | Arithmetic overflow: bit 0 = 1

Ⓡ EPC, 32 bits, loadable from ALU output

■ Control signals

Ⓡ CauseWrite

Ⓡ EPCWrite

Ⓡ 1 bit signals IntCause

Ⓡ Exception address, C0000000_{hex}

■ PC Mux extension

Ⓡ ALU

Ⓡ Branch

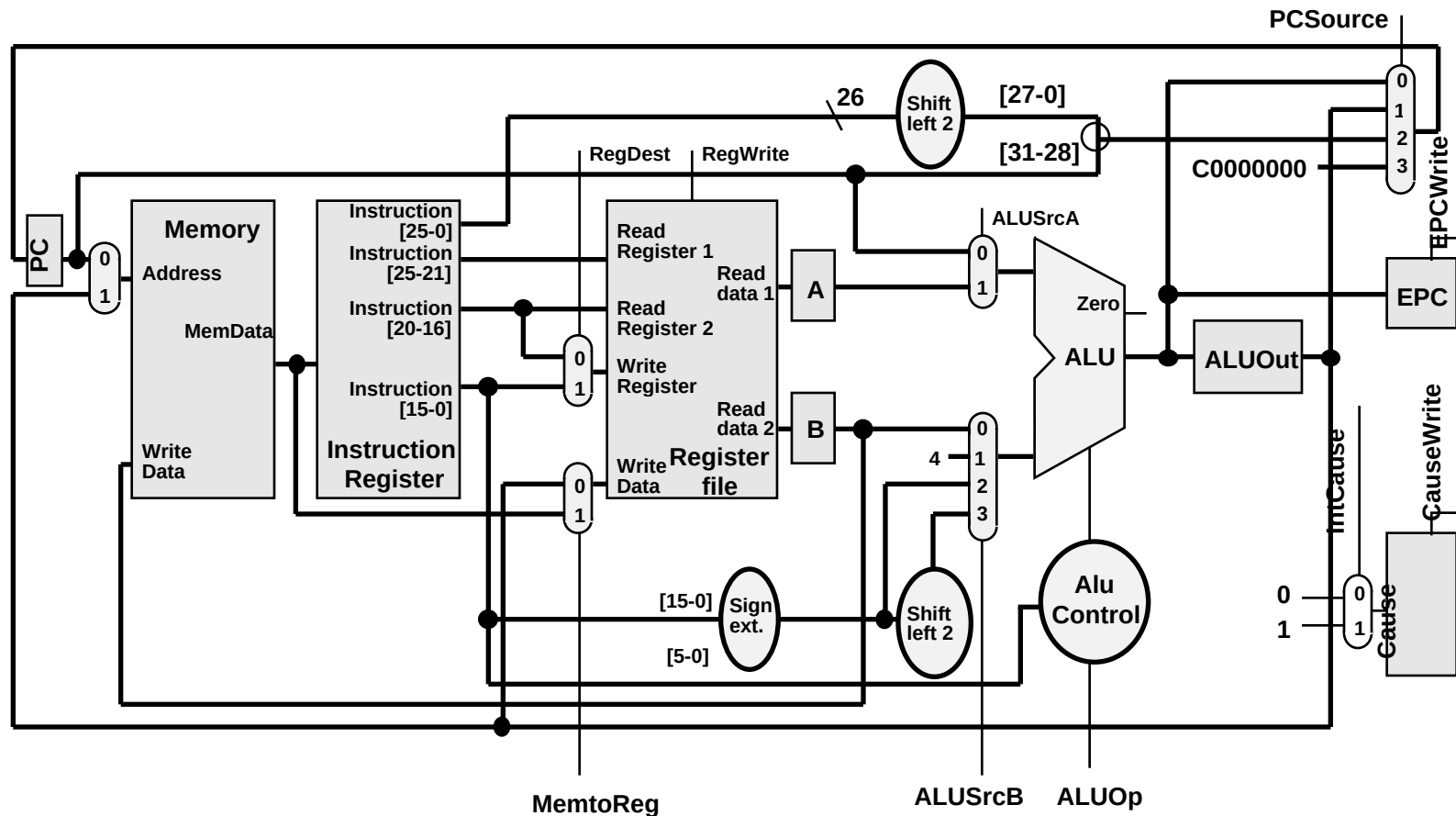
Ⓡ Jump

Ⓡ Exception

■ PC decrement by 4 to get address of current instruction

Extended architecture

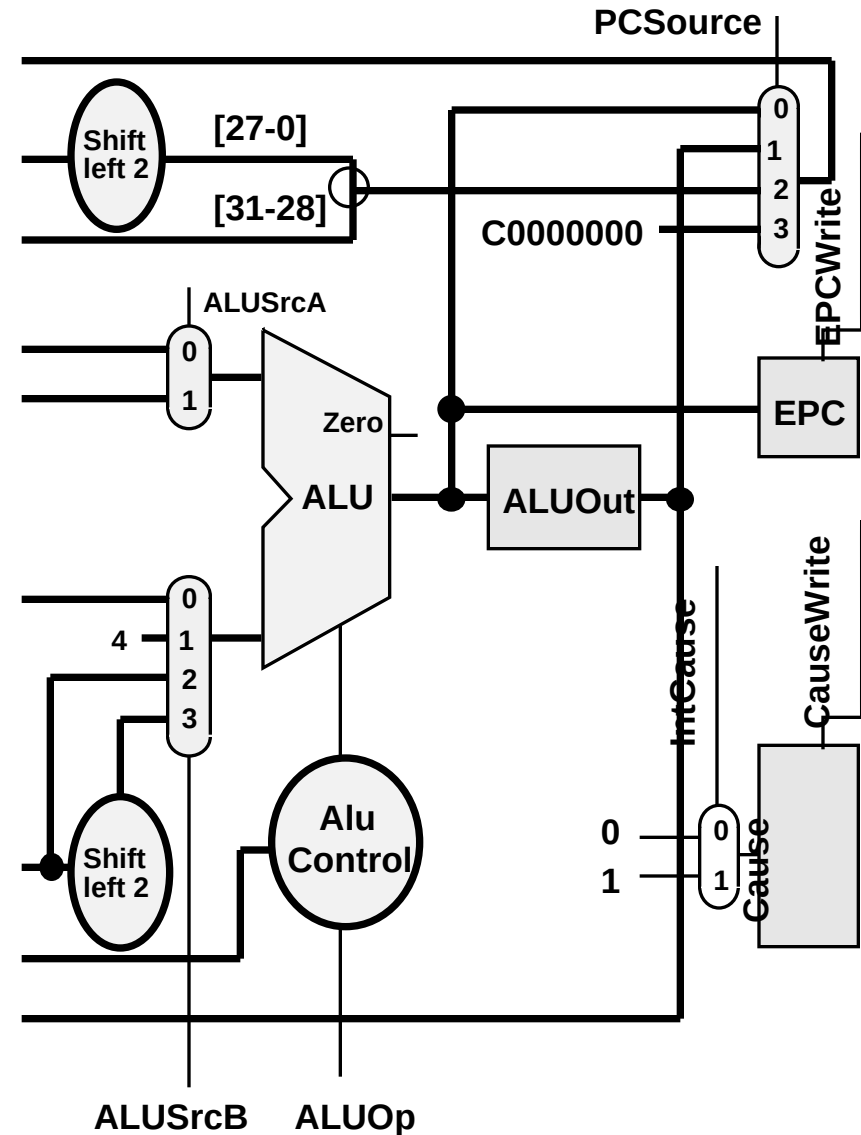
The Processor



Extended architecture

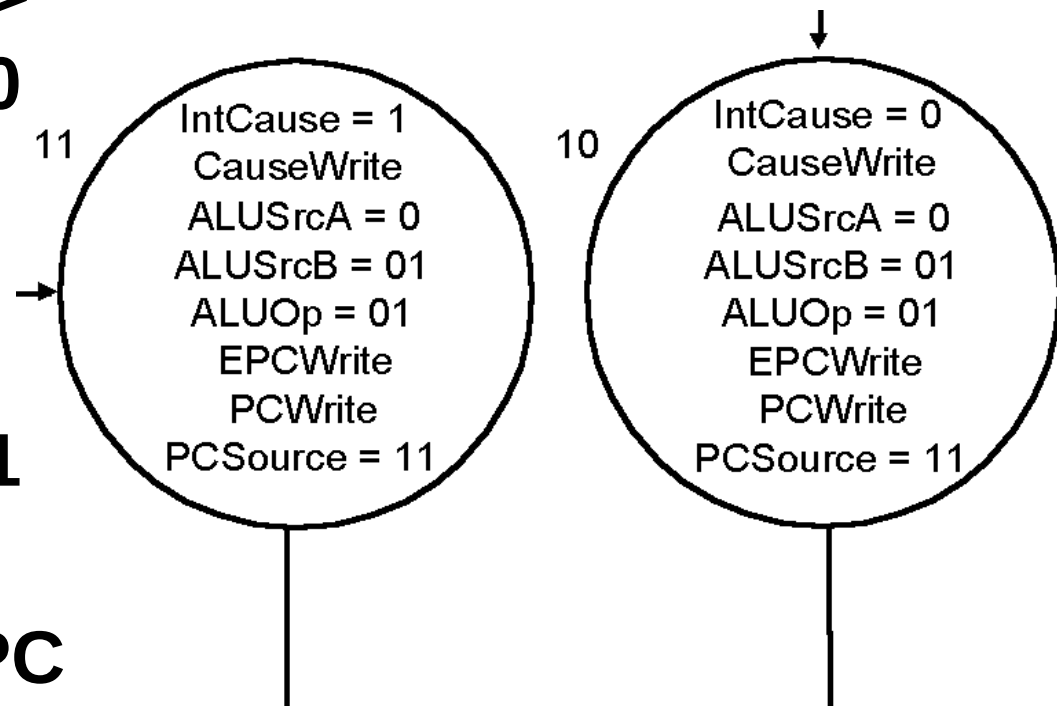
The Processor

- Cause register
- Cause write
- EPC
- EPCWrite
- PCSource extension

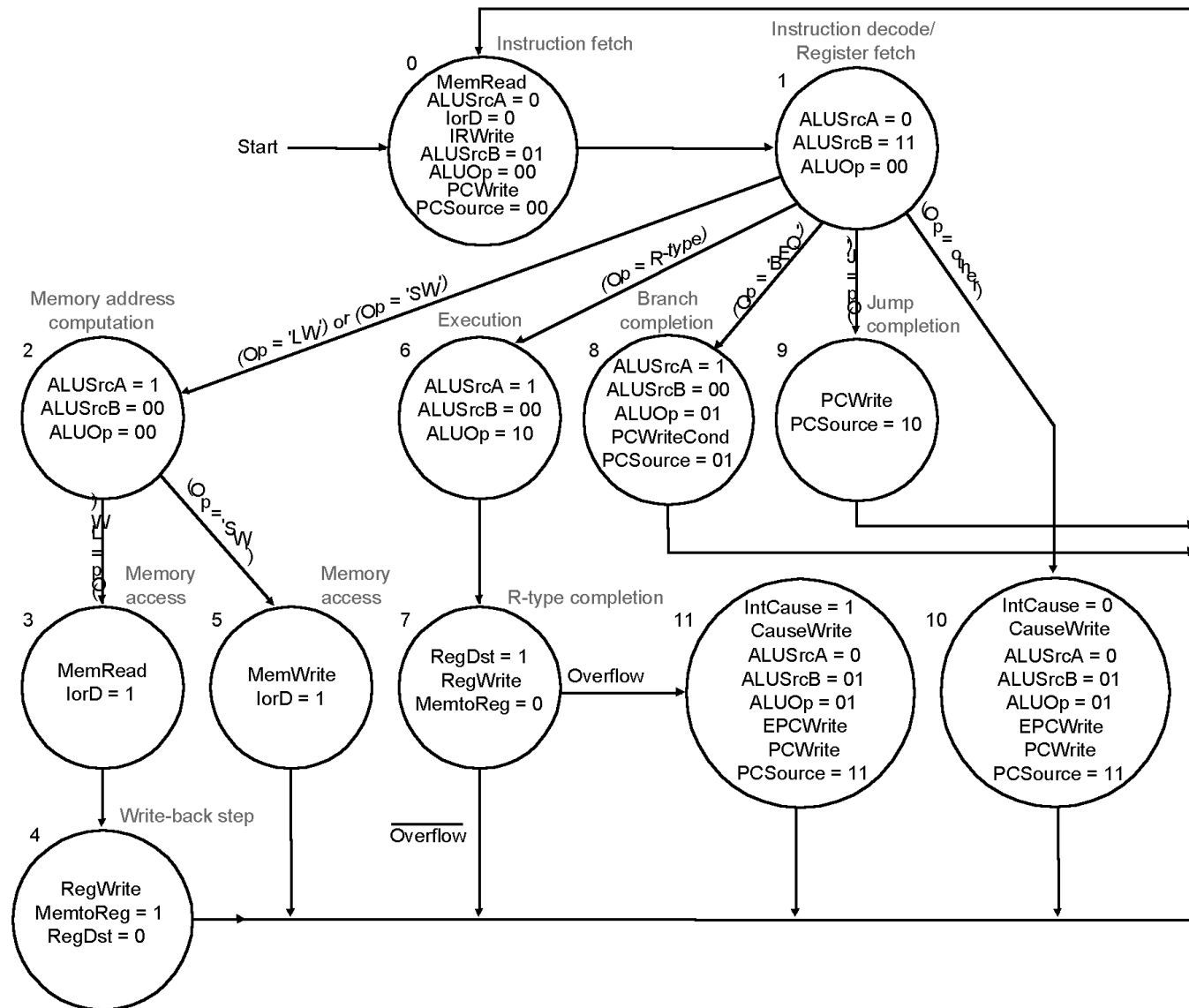


Exception handling

- Undefined instruction > IntCause = 0 (State 10)
- Arithmetic overflow > IntCause = 1 (State 11)
- Load the EPC
- Load PC with interrupt register



State machines with exception



Implementation options

The Processor

Initial representation	Finite state diagram	Microprogram
Sequencing control	Explicit next state function	Microprogram counter + dispatch ROMS
Logic representation	Logic equations	Truth tables
Implementation technique	Programmable logic array	Read only memory

```

graph TD
    FSD[Finite state diagram] --> ENSF[Explicit next state function]
    FSD --> MPC[Microprogram counter + dispatch ROMS]
    MP[Microprogram] --> ENSF
    MP --> MPC
    ENSF --> LE[Logic equations]
    ENSF --> TT[Truth tables]
    MPC --> LE
    MPC --> TT
    LE --> PLA[Programmable logic array]
    LE --> ROM[Read only memory]
    TT --> PLA
    TT --> ROM
  
```

Summary

■ Microprogramming

- ④ gives flexibility
- ④ Is not necessarily faster than a RISC approach

■ Hardwired control

- ④ Is fast
- ④ Complex to design
- ④ Not flexible

■ But

- ④ We have more powerful design tools
- ④ Flexible specification
- ④ Automatic synthesis